

Atividade Prática: Camada de Aplicação e Transporte

“Chat via UDP”: Confirmação de Entrega e Reenvio Manual em Nível de Aplicação

Disciplina: Redes de Computadores

Visão Geral da Atividade

Nesta atividade prática, você implementará um serviço simplificado de mensagens instantâneas operando sobre o protocolo **UDP** (`SOCK_DGRAM`). O canal de rede simulará perda artificial de pacotes, exigindo que a própria aplicação implemente mecanismos de identificação de mensagens, recibos de entrega (*Application-level ACKs*) e rotina de retransmissão de mensagens pendentes.

1 Contextualização

Ao contrário do TCP, que estabelece conexão e garante entrega de fluxo de bytes de forma transparente ao sistema operacional, o UDP opera sob o paradigma de *melhor esforço*. Datagramas podem ser descartados em roteadores intermediários sem que o emissor tome ciência disso.

Em aplicações reais (como mensageiros instantâneos que utilizam UDP para contornar gargalos de handshake), a confirmação de que uma mensagem chegou ao destinatário é tratada diretamente na **Camada de Aplicação**. O objetivo desta prática é reproduzir essa experiência: permitir que o usuário envie mensagens, veja quais foram confirmadas pelo destinatário e consiga reenviar aquelas que ficaram pelo caminho.

2 Especificação do Protocolo de Aplicação

As mensagens trocadas entre remetente e receptor devem adotar um formato de texto com separador delimitado (`|`), compreendendo dois tipos:

1. Mensagem de Texto (MSG):

- Formato: `MSG|<ID>|<CONTEUDO>`
- Exemplo: `MSG|1|Oi, tudo bem?`

2. Recibo de Entrega (DELIVERED):

- Formato: `DELIVERED|<ID>`
- Exemplo: `DELIVERED|1`

3 Comportamento dos Componentes

3.1 1. Servidor / Destinatário (`chat_receiver.py`)

- Aguarda datagramas na porta pré-definida.
- Submete cada datagrama recebido a uma taxa de perda aleatória (`DROP_RATE = 0.4`). Se descartado, o pacote é ignorado.
- Se processado com sucesso, extrai o ID e o texto, imprime a mensagem na tela e envia de volta ao remetente o recibo `DELIVERED|<ID>`.

3.2 2. Cliente / Remetente (chat_sender.py)

- Mantém um dicionário/histórico de mensagens enviadas: `pending_messages = {id: texto}`.
- O usuário digita mensagens no terminal:
 - Cada nova mensagem recebe um ID sequencial incremental e é enviada ao servidor.
 - Ela é registrada inicialmente no status [Pendente (Check Cinza)].
- Uma *thread* de escuta recebe os pacotes `DELIVERED|<ID>` vindos do servidor e remove a mensagem correspondente da lista de pendências, exibindo [Entregue (Check Azul)].
- Comando especial `/reenviar`: Itera sobre todas as mensagens que continuam em `pending_messages` e as envia novamente via UDP.
- Comando especial `/status`: Exibe o resumo de mensagens pendentes versus confirmadas.

4 Código-Base Fornecido (Esqueletos)

4.1 Esqueleto do Destinatário (chat_receiver.py)

```
1 import socket
2 import random
3
4 HOST = '0.0.0.0'
5 PORT = 5001
6 DROP_RATE = 0.4 # 40% de perda simulada no canal
7
8 def run_chat_receiver():
9     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
10         s.bind((HOST, PORT))
11         print(f"[Chat Server] Online na porta {PORT} (Drop Rate: {DROP_RATE * 100}%)
12         ...")
13
14         while True:
15             data, addr = s.recvfrom(1024)
16             if random.random() < DROP_RATE:
17                 print("[CANAL] Pacote descartado artificialmente!")
18                 continue
19
20             raw_message = data.decode('utf-8')
21             # TODO 1: Fazer o split da mensagem delimitada por '|'
22             # TODO 2: Verificar se a mensagem é do tipo 'MSG'
23             # TODO 3: Extrair o ID da mensagem e o texto do usuário
24             # TODO 4: Exibir no terminal a mensagem recebida e o ID correspondente
25             # TODO 5: Montar o pacote de recibo no formato 'DELIVERED|<ID>'
26             # TODO 6: Enviar o recibo de volta para a origem usando s.sendto(...,
27             addr)
28
29 if __name__ == '__main__':
30     run_chat_receiver()
```

4.2 Esqueleto do Remetente (chat_sender.py)

```
1 import socket
2 import threading
3
4 TARGET_IP = '127.0.0.1'
5 PORT = 5001
6
7 pending_messages = {}
8 msg_counter = 1
```

```

9 lock = threading.Lock()
10
11 def listen_receipts(sock):
12     while True:
13         try:
14             data, _ = sock.recvfrom(1024)
15             raw = data.decode('utf-8')
16             # TODO 1: Parsing do recibo
17             # TODO 2: Checar se e 'DELIVERED'
18             # TODO 3: Extrair o ID
19             # TODO 4: Remover de pending_messages e exibir confirmacao
20         except Exception:
21             break
22
23 def run_chat_sender():
24     global msg_counter
25     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
26         listener = threading.Thread(target=listen_receipts, args=(s,), daemon=True)
27         listener.start()
28
29         while True:
30             try:
31                 user_input = input("Digite uma mensagem: ").strip()
32                 if not user_input:
33                     continue
34                 if user_input == "/status":
35                     # TODO 5: Exibir mensagens pendentes
36                     continue
37                 if user_input == "/reenviar":
38                     # TODO 6: Reenviar mensagens pendentes
39                     continue
40
41                 # TODO 7 a 10: Cadastrar mensagem, enviar MSG|<ID>|<TXT> e exibir
42                 status
43             except KeyboardInterrupt:
44                 break
45
46 if __name__ == '__main__':
47     run_chat_sender()

```

5 Instruções para Entrega

Deverá ser entregue um repositório no github contendo:

- a. **Código-fonte:** Os arquivos `chat_sender.py` e `chat_receiver.py` devidamente comentados.
- b. **Relatório Sucinto (PDF):**
 - Breve descrição da estratégia adotada para controlar as pendências e os recibos.
 - **Captura de tela 1:** O cliente enviando mensagens onde ocorre descarte proposital pelo canal (status pendente visível no `/status` e descarte indicado no servidor).
 - **Captura de tela 2:** A execução do comando `/reenviar` destravando e confirmando a entrega de todas as mensagens pendentes.
 - Discussão curta sobre a diferença entre fazer esse controle na camada de aplicação versus a confiabilidade nativa do TCP.

Não é permitido o uso de sockets TCP (`SOCK_STREAM`).